Vidyavardhini's College of Engineering and Technology, Vasai
Department of Computer Science & Engineering (Data Science)

Experiment 8

NAME: Vedant Naik
ROLLNO: 41
CLASS: SE-2

Title

Building a Simple Counter App using React Hooks. Use React Hooks (useState, useEffect) to manage state and side effects.

Theory:

React is a popular JavaScript library used for building interactive user interfaces. Traditionally, React applications used **class components** to manage state and lifecycle methods. With the introduction of **React Hooks**, functional components can now manage state and side effects more efficiently.

1. React Hooks

Hooks are special functions in React that allow developers to "hoo

k into" React features such as state and lifecycle methods from functional components. Hooks improve code readability, reusability, and reduce complexity.

2. useState Hook

The useState hook is used to manage **state** in functional components. State refers to data that can change over time and affects what is rendered on the screen.

In a counter application, the counter value is stored as state. Whenever the state changes, React automatically re-renders the component to reflect the updated value.

3. useEffect Hook

The useEffect hook is used to handle **side effects** in functional components. Side effects include:

- Updating the document title
- Fetching data

- Logging values
- Running code after component render

In this experiment, useEffect is used to update the browser title whenever the counter value changes, demonstrating how side effects are handled in React.

4. **Component Re-rendering**

Whenever the state changes using useState, React re-renders the component. The useEffect hook runs after rendering, ensuring that side effects are synchronized with the component's state.

Algorithm / Procedure

1. Start the experiment.
2. Create a new React application using create-react-app.
3. Open the project folder in VS Code.
4. Create a functional component for the Counter App.
5. Initialize counter state using useState.
6. Add buttons to increment and decrement the counter value.
7. Use useEffect to update the document title when the counter changes.
8. Display the counter value dynamically on the webpage.
9. Run the React application in the browser.
10. Stop the experiment.

Program / Code:

Counter Component (App.js) import React, {

```
useState, useEffect } from "react";


function App() {   const [count, setCount]
= useState(0);   useEffect(()
=> {     document.title = `Count:
${count}`;
  }, [count]);


  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h2>Simple Counter App</h2>
      <h1>{count}</h1>

      <button onClick={() => setCount(count + 1)}>Increment</button>
        
      <button onClick={() => setCount(count - 1)}>Decrement</button>
</div>
  );
}


export default App;
```

Scenario based questions:

Develop a simple React Counter Application using React Hooks. The application should display a counter value starting from zero and provide buttons to increase, decrease, and reset the counter. Use the useState hook to manage the counter state and the useEffect hook to perform a side effect whenever the counter value changes (such as displaying a message in the console or updating the page title). The application should update the UI dynamically without page reload

Output:

Conclusion

Thus, a simple counter application was successfully developed using React Hooks. The experiment demonstrates the use of useState for state management and useEffect for handling side effects. This approach simplifies component logic and improves code readability in modern React applications.

Questions:

1.What are React Hooks?

Hooks are special functions that allow you to "hook into" React features like state and lifecycle methods without writing a class.

Before hooks, functional components were "stateless" (they couldn't remember data). Now, they can do everything a class could do, but with much cleaner code.

- **useState**: Allows a component to remember and update data.

- **useEffect**: Handles "side effects," like fetching data from an API or setting up a subscription.

2.How does React re-render components?

A re-render happens whenever the "state" of a component changes. React follows a specific process to ensure the UI stays in sync with the data:

1. **Trigger:** A change occurs in the component's **State** or its **Props** (data passed from a parent).
2. **Render:** React calls your function component again to see what the UI *should* look like now.
3. **Commit:** React calculates the difference between the old UI and the new UI (using the Virtual DOM) and applies only the necessary changes to the actual browser screen.

3.How does React re-render components?

To make re-rendering efficient, React uses something called the **Virtual DOM**.

Instead of touching the actual browser screen every time something tiny changes (which is slow), React creates a lightweight copy of the UI in memory.

- It compares the new Virtual DOM with the previous one—a process called **Diffing**.
- It identifies exactly which parts changed (e.g., just one list item) and updates **only** those specific elements in the real DOM. This "surgical" update is why React feels so fast.